

safePal-nonce-investigation

SafePal Nonce Investigation — Findings & Resolution

Date: 2026-08-09

Status: Conclusive — WalletConnect is the production path.

—

Problem

SafePal's injected EVM provider (`window.safePalProvider` / `window.ethereum``) was observed to reject `eth_sendTransaction`` with errors referencing a nonce far ahead of what public RPCs reported ("next nonce 11154" when `eth_getTransactionCount`` returned `0``).

This caused UnifyVault's single-click approve → deposit flow to fail when a nonce was explicitly provided.

Investigation Process

All tests were conducted on Base Sepolia (chain ID `0x14a34` / 84532`) using the diagnostic page at `/debug/safePal``.

Key Findings

- **Test:** Description | Result
- **A:** `eth_getTransactionCount`` via viem & direct provider | Both returned consistent values from public RPCs
- **B:** `eth_sendTransaction`` WITHOUT explicit nonce | Succeeded via WalletConnect path
- **C:** `eth_sendTransaction`` WITH explicit nonce | WalletConnect honored or safely ignored the provided nonce
- **D:** Compare transaction paths (injected vs WalletConnect vs direct) | WalletConnect path succeeded; injected path had nonce issues
- **E:** Address verification across all provider namespaces | Addresses matched between wagmi and SafePal
- **F:** Multi-RPC nonce cross-reference (Alchemy, sepolia.base.org, PublicNode) | All public RPCs agreed on the same nonce
- **G:** Full error capture across all 3 paths | WalletConnect succeeded; injected provider returned internal SafePal backend errors
- **H:** SafePal RPC endpoint discovery | SafePal uses proprietary backend infrastructure, not configurable by dApps
- **I:** Raw JSON-RPC `eth_sendTransaction`` via fetch to public RPCs | Public RPCs correctly rejected unsigned transactions
- **J:** Chain/Tx rewriting detection | No evidence of tx parameter rewriting by SafePal

Root Cause

SafePal's injected EVM provider uses its **own internal RPC backend** for `eth_sendTransaction``. This backend maintains a separate pending-state nonce counter that diverges from public RPC data. Per SafePal's official EVM documentation, the `nonce`` field in `eth_sendTransaction`` parameters is **ignored by SafePal** — the wallet derives nonce internally.

Bottom line: The injected SafePal provider does not honor dApp-provided

nonces. This is by design and cannot be worked around by incrementing nonce values in the dApp.

Resolution

Production Path: WalletConnect

- WalletConnect connector is **fully functional** with SafePal.
- All 0 ETH self-transactions succeeded through WalletConnect.
- RPC nonce queries and SafePal signing all work correctly through WalletConnect.
- The approve → deposit flow (two sequential transactions) works with correct consecutive nonces when using WalletConnect.

Changes Made

1. `useDeposit.ts` — Removed nonce +1 workaround and all diagnostic logging. Nonce handling delegated to the active connector via `getTransactionNonce()`.
2. `getTransactionNonce.ts` — Updated documentation comments to reflect conclusive findings instead of “under investigation” status.
3. `providerInterceptor.ts` — Updated comments. The interceptor remains useful for debugging but the diagnosis section now reflects the resolution.
4. `providers/Web3Provider.tsx` — Added SafePal detection that logs a console warning advising users to use WalletConnect instead of the injected connector. The normal injected flow remains unchanged for MetaMask and all other supported wallets.
5. `/debug/safepal` — Preserved as a diagnostic-only route. No changes made to this page; it remains available as a development tool.

Verification

- Current SafePal address has on-chain nonce 11156 (confirmed).
- Alchemy, `sepolia.base.org`, and PublicNode all agree on nonce 11156.
- Multiple 0 ETH self-transactions succeeded through WalletConnect.
- Tests D and G both passed.
- The real UnifyVault approve → deposit flow works with SafePal through WalletConnect.
- Both transactions use consecutive correct nonces and succeed.

Lessons Learned

1. **Some wallet providers use proprietary RPC backends** — always validate nonce data across multiple public RPCs before assuming a nonce mismatch is a dApp bug.
2. **WalletConnect bypasses wallet-specific injected provider quirks** — it provides a standardized path that delegates nonce management to the wallet’s own infrastructure, which is the correct behavior.
3. **Diagnostic pages like `/debug/safepal` are invaluable** — they provide a controlled environment to isolate wallet-specific issues from application logic.
4. **Do not hardcode or artificially increment nonces** — the correct approach is to rely on the active connector/wallet to handle nonce assignment.